Architectural Patterns

A visual field guide to four classic software books

C++20 code | diagrams | traps to avoid | memory-management tips

ZOOM OUT

Design Patterns
Gang of Four
OBJECTS
How do objects work together without being welded together?
Factory | Observer | Decorator

Clean Architecture
Robert C. Martin
COMPONENTS
Which way should source dependencies point?
SOLID | Dependency Rule | Ports

Game Programming Patterns
Robert Nystrom
RUNTIME
How do I stay fast and predictable on every single frame?
Game loop | Pool | Data locality

Designing Data-Intensive Applications
Martin Kleppmann
DISTRIBUTED DATA
How does data stay correct when machines and networks fail?
Logs | Replication | LSM-trees

Four ideas show up in every one of these books

1. **Control the direction of dependencies**
2. **Separate policy from mechanism**
3. **Make ownership and state explicit**
4. **Design for change and for failure**

KEY IDEA the one thing to remember | WATCH OUT common traps | MEMORY TIP who owns what

This guide is my own condensed synthesis of the ideas in these books, not a reproduction of their text. The code is original, compact C++20 meant to teach each idea (I have not compiled it). Check the books for exact wording and page references.

1 Design Patterns

Gang of Four | Object level | How do objects work together without being welded together?

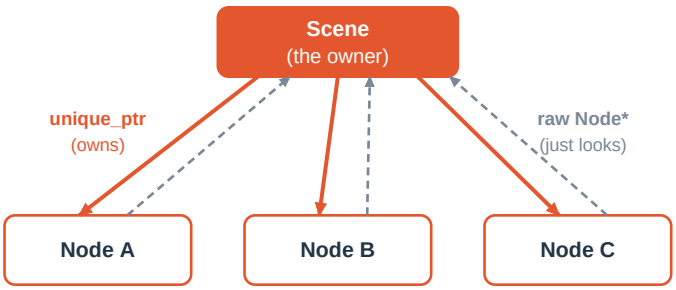
KEY IDEA
Three rules sit behind all 23 patterns: **program to an interface**, **favor composition over inheritance**, and **encapsulate what varies**.

Creational (who builds it?)	Structural (how is it assembled?)	Behavioral (how do they talk?)	
Factory Method	Adapter	Observer	Template Method
Abstract Factory	Bridge	Mediator	Visitor ●
Builder	Composite	Chain of Resp.	Iterator ●
Prototype ●	Decorator	Command ●	Memento
Singleton ●	Facade	State	Interpreter
	Flyweight	Strategy ●	
	Proxy		

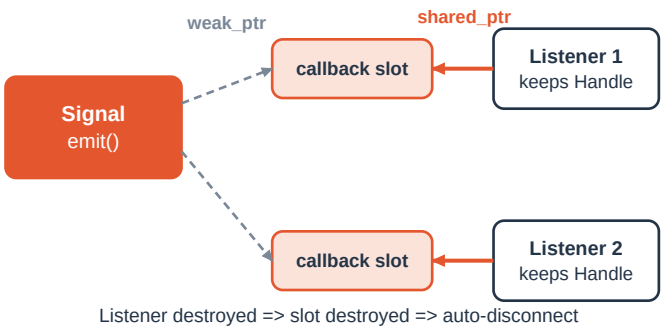
THE 23 PATTERNS AT A GLANCE (yellow dot = modern C++ already does much of the work)

Strategy is now `std::function` or a template parameter, Visitor is `std::variant` plus `std::visit`, Command is often a lambda, Iterator is ranges, Prototype is copy semantics, and Singleton is a function-local static. Learn the pattern for its *name*, then implement it with the modern tool.

COMPOSITE: parent owns, children only look back



OBSERVER: subscribers own the link, the signal only watches



Listener destroyed => slot destroyed => auto-disconnect

WHO OWNS WHOM: in C++ every pattern is secretly an ownership decision

Factory Method with a registry: add a type without editing an if/else chain

```
struct Shape { virtual ~Shape() = default; virtual double area() const = 0; };

class ShapeFactory {
public:
    using Maker = std::function<std::unique_ptr<Shape>(double)>;
    static ShapeFactory& instance() { static ShapeFactory f; return f; }     // thread-safe init since C++11
    void add(std::string key, Maker m) { makers_[std::move(key)] = std::move(m); }
    std::unique_ptr<Shape> make(const std::string& key, double size) const {
        auto it = makers_.find(key);
        return it == makers_.end() ? nullptr : it->second(size);            // caller owns the result
    }
private:
    std::unordered_map<std::string, Maker> makers_;
};
// at startup: ShapeFactory::instance().add("circle", [](double r) { return std::make_unique<Circle>(r); });
```

Observer that cleans up after itself: when the listener dies, it is disconnected automatically

```
template <class... Args>
class Signal {
public:
    using Fn = std::function<void(Args...)>;
    using Handle = std::shared_ptr<Fn>;  // the subscriber keeps this alive to stay connected
    [[nodiscard]] Handle connect(Fn f) { auto h = std::make_shared<Fn>(std::move(f)); slots_.push_back(h); return h; }
    void emit(Args... a) {
        auto snapshot = slots_;                                     // handlers may connect or disconnect mid-emit
        for (auto& w : snapshot) if (auto h = w.lock()) (*h)(a...);
        std::erase_if(slots_, [](auto& w) { return w.expired(); });         // C++20
    }
private:
    std::vector<std::weak_ptr<Fn>> slots_;                          // we observe, we never own
};
```

Visitor in four lines: a closed set of types, and the compiler checks every branch

```
struct Circle { double r; };   struct Rect { double w, h; };
using Shape = std::variant<Circle, Rect>;                           // value semantics: no heap, no virtuals

double area(const Shape& s) {
    return std::visit([](const auto& v) {
        if constexpr (std::is_same_v<std::decay_t<decltype(v)>, Circle>) return 3.14159265 * v.r * v.r;
        else                                                              return v.w * v.h;
    }, s);
}
```

WATCH OUT

- **Singleton = global state.** Hidden dependencies, untestable code, and destruction-order bugs at exit. Create things once in `main` and pass them in.
- **Self-registering globals** run in an unspecified order across files (static-init-order fiasco) and can be dropped by the linker.
- **Observer traps:** a listener destroyed while still registered, a handler that edits the subscriber list during notification, and update loops.
- **Slicing and virtual calls:** passing a polymorphic type by value cuts off the derived part; calling virtuals in a constructor runs the base version.

MEMORY TIP

Exclusive owner = `unique_ptr`. Back-pointers and observers = non-owning (`T*`, `T&`, `weak_ptr`). Keep `shared_ptr` for truly shared lifetimes. Prefer `make_unique`/`make_shared` to naked `new`. Destructors must never throw: catch inside them.

2 Clean Architecture

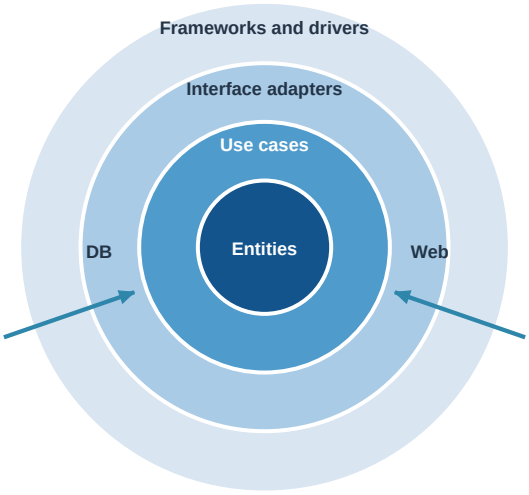
Robert C. Martin | Component level | Which way should source dependencies point?

KEY IDEA

Business rules must not know about databases, frameworks or UIs. Those are **details that plug into the policy**, never the other way around. A good architecture keeps decisions open as long as possible.

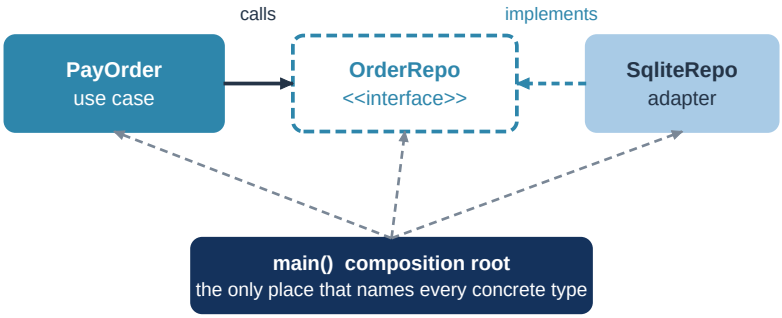
SOLID IN FIVE WORDS EACH

S One reason to change	**O** Extend, do not edit	**L** Subtypes keep the promise	**I** Small interfaces	**D** Depend on abstractions



Source dependencies point INWARD only

Dependency inversion: control goes one way, the source dependency the other



wires the pieces together at startup

PayOrder never mentions SQLite, so you can test it with a fake.

THE DEPENDENCY RULE: outer layers know about inner layers, never the reverse

- **Components:** no cycles between them (ADP). In C++ a cycle between CMake targets will not even build, so the toolchain enforces it for you.
- **Boundaries carry plain data:** small structs passed by value, never ORM rows or framework types.
- **Humble object:** keep GUI and hardware code paper-thin so everything interesting is unit-testable.

A use case that owns its ports. SQL, HTTP and Stripe live elsewhere and are wired in `main`

```
struct Order { int id{}; double total{}; bool paid{false}; };  // domain: depends on nothing

struct OrderRepo      { virtual ~OrderRepo() = default;  // ports: declared by the use-case layer
                        virtual std::optional<Order> find(int id) = 0;
                        virtual void save(const Order&) = 0; };
struct PaymentGateway { virtual ~PaymentGateway() = default; virtual bool charge(double amount) = 0; };
enum class PayResult { Ok, NotFound, AlreadyPaid, Declined };

class PayOrder {  // interactor = application business rule
public:
    PayOrder(OrderRepo& repo, PaymentGateway& gw) : repo_(repo), gw_(gw) {}         // non-owning references
    PayResult operator()(int id) {
        auto o = repo_.find(id);
        if (!o)      return PayResult::NotFound;
        if (o->paid) return PayResult::AlreadyPaid;
        if (!gw_.charge(o->total)) return PayResult::Declined;
        o->paid = true;  repo_.save(*o);  return PayResult::Ok;
    }
private:
    OrderRepo& repo_;  PaymentGateway& gw_;
};

int main() {                                                                        // composition root
    SqliteOrderRepo repo{"shop.db"};  StripeGateway gw{api_key()};  // owners first => destroyed LAST
    PayOrder pay{repo, gw};
    return run_http_server(pay);
}
```

WATCH OUT

- **Over-layering:** a mapper at every boundary for plain CRUD, or an interface for every class. Add boundaries only where change is really likely.
- **Anemic entities** (just getters/setters) while the rules leak into controllers; **ORM or JSON types** leaking through use-case signatures.
- **Virtual calls in hot loops:** a template (`template <class Repo> class PayOrderT`) gives the same inversion with static dispatch.
- **pimpl gotcha:** `std::unique_ptr<Impl>` with an incomplete type needs the destructor defined in the .cpp file, or it will not compile.

MEMORY TIP

The composition root owns long-lived objects and hands out references; declare owners before users so destruction runs in reverse dependency order. Pass request/response models by value or move. Take `std::span` and `std::string_view` as parameters but never store them. Do not thread `shared_ptr` through every layer to avoid thinking about lifetimes.

3 Game Programming Patterns

Robert Nystrom | Runtime level | How do I stay fast and predictable on every frame?

KEY IDEA

A frame has a hard deadline (16.6 ms at 60 Hz) and **cache misses cost more than clever algorithms**. These patterns apply to any latency-sensitive C++: audio, trading, robotics, embedded.

Game Loop
Fixed step for simulation, variable for rendering, interpolate between.

Component
Split the god-object into independent parts (physics, render, AI).

Dirty Flag
Defer derived work (transforms, bounds) until someone needs it.

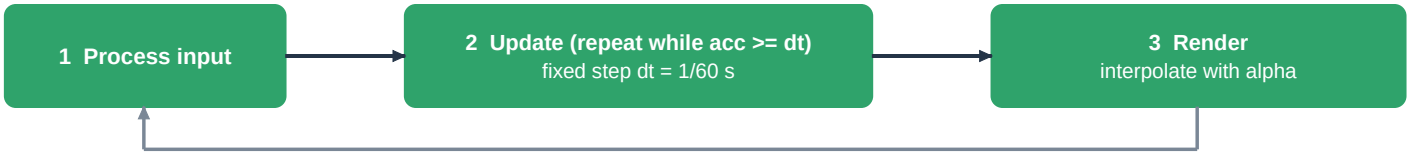
Event Queue
Decouple sender and receiver in time, not just in type.

Double Buffer
Write to a back buffer, swap, so readers never see half-updated state.

Data Locality
Lay data out for the cache; process by system, not by entity.

Object Pool
Preallocate fixed slots and reuse them; no malloc in the frame.

Spatial Partition
Grids, quadtrees, BVHs turn O(n squared) queries into local ones.



Real elapsed time goes into an accumulator; each full dt is simulated; the leftover fraction is alpha.

GAME LOOP: simulation time is decoupled from frame rate

Fixed timestep with interpolation

```
using Clock = std::chrono::steady_clock;                        // monotonic: never time with system_clock
constexpr double kDt = 1.0 / 60.0;  // fixed step => deterministic physics, reproducible bugs
double acc = 0.0;  auto prev = Clock::now();
while (running) {
    auto now = Clock::now();
    acc += std::min(std::chrono::duration<double>(now - prev).count(), 0.25);  // clamp: avoids the "spiral of death"
    prev = now;
    process_input();
    while (acc >= kDt) { update(kDt); acc -= kDt; }
    render(acc / kDt);                                          // alpha in [0,1): blend the last two states
}
```

Hot loop only needs x and vx. Green = needed, grey = dragged into the cache for nothing.

Array of structs

Struct of arrays

Same work, but every cache line is full of useful data: fewer misses, and the CPU can vectorize.

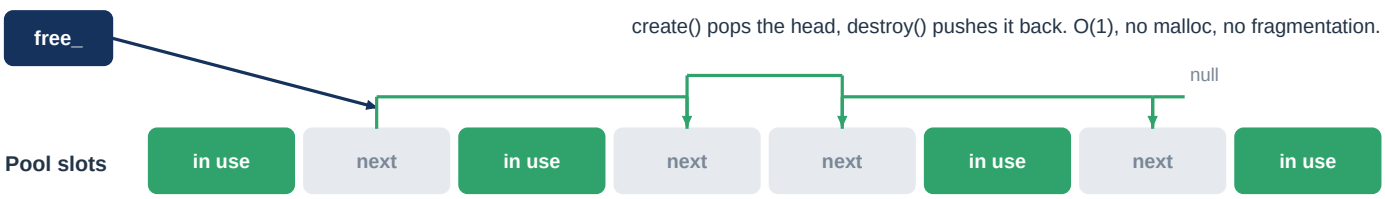
Generational handle = (slot index, generation). Slot reused => generation bumps => old handles fail safely.

DATA LOCALITY: array-of-structs vs struct-of-arrays

Struct-of-arrays, plus generational handles instead of raw pointers

```
struct Particles {
    std::vector<float> x, y, vx, vy, life;
    void step(float dt) {
        for (std::size_t i = 0; i < x.size(); ++i) { x[i] += vx[i]*dt; y[i] += vy[i]*dt; life[i] -= dt; }
    }
    void erase_unordered(std::size_t i) {                       // O(1): swap with last, arrays stay dense
        std::size_t n = x.size() - 1;
        x[i] = x[n]; y[i] = y[n]; vx[i] = vx[n]; vy[i] = vy[n]; life[i] = life[n];
        x.pop_back(); y.pop_back(); vx.pop_back(); vy.pop_back(); life.pop_back();
    }
};

struct Handle { std::uint32_t index, gen; };  // survives reallocation, detects use-after-free
template <class T> class Slots {
    std::vector<T> items_;  std::vector<std::uint32_t> gen_, free_;
public:
    Handle add(T v) {
        std::uint32_t i;
        if (free_.empty()) { i = std::uint32_t(items_.size()); items_.push_back(std::move(v)); gen_.push_back(0); }
        else               { i = free_.back(); free_.pop_back(); items_[i] = std::move(v); }
        return {i, gen_[i]};
    }
    T* get(Handle h) { return h.index < items_.size() && gen_[h.index] == h.gen ? &items_[h.index] : nullptr; }
    void remove(Handle h) { if (get(h)) { ++gen_[h.index]; free_.push_back(h.index); } }   // stale handles fail
};
```



create() pops the head, destroy() pushes it back. O(1), no malloc, no fragmentation.

OBJECT POOL: free slots form a linked list inside the slots themselves

Object pool with an intrusive free list

```
template <class T, std::size_t N>
class Pool {  // fixed capacity, zero heap traffic after construction
    union Slot { alignas(T) unsigned char storage[sizeof(T)]; Slot* next; };
    Slot slots_[N];  Slot* free_ = nullptr;
public:
    Pool() { for (std::size_t i = 0; i < N; ++i) { slots_[i].next = free_; free_ = &slots_[i]; } }
    template <class... A> T* create(A&&... a) {
        if (!free_) return nullptr;                              // full: drop, evict or grow? decide on purpose
        Slot* s = free_;  free_ = s->next;
        return new (s->storage) T(std::forward<A>(a)...);        // placement new
    }
    void destroy(T* p) { p->~T(); auto* s = reinterpret_cast<Slot*>(p); s->next = free_; free_ = s; }
};
```

WATCH OUT

- **Variable timestep** makes physics nondeterministic; a missing clamp lets one long frame trigger ever-longer catch-up.
- **Pools hide use-after-free** (a destroyed slot still looks valid), never destruct objects still alive at pool teardown, and overflow the stack if `Pool` is huge.
- **Events carry IDs or handles, never raw pointers.** Queues also create feedback loops and surprising ordering.
- **Do not optimise cache layout without measuring.** Profile first and keep the simple version as your baseline.

MEMORY TIP

Allocate at load time, not inside the frame. Use per-frame arenas (reset a bump pointer), `reserve()` vectors, and `alignas(64)` on hot per-thread data to avoid false sharing. Keep hot and cold fields in separate structs. Avoid `shared_ptr` and `std::function` in inner loops.

4 Designing Data-Intensive Applications

Martin Kleppmann | Distributed data | How does data stay correct when machines fail?

KEY IDEA

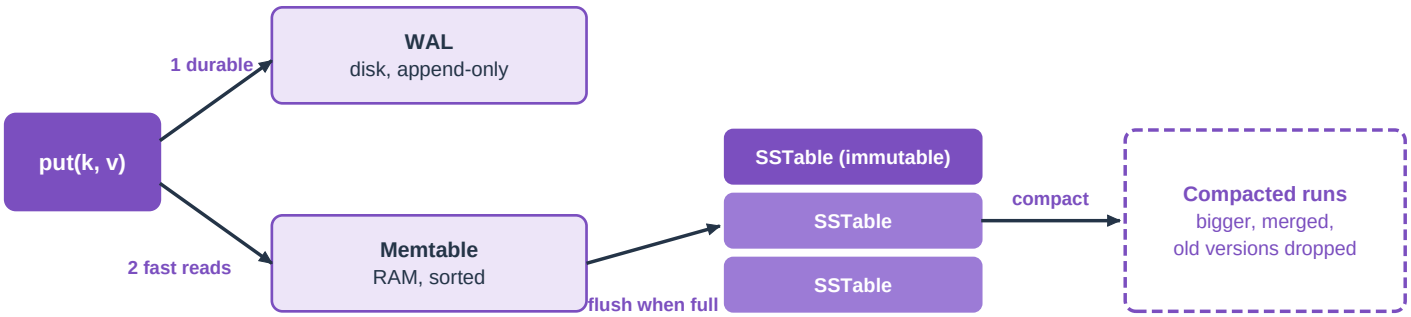
Every design is a trade-off between consistency, availability, latency and simplicity. The unifying idea is the **append-only log**: write-ahead logs, replication streams, Kafka topics and change feeds are all ordered sequences of events.

Reliable
Keeps working correctly when hardware, software and people fail.

Scalable
Has a plan for growth. Measure percentile latency, not averages.

Maintainable
Easy to operate, simple to understand, safe to evolve.

LSM-tree write path: sequential I/O only



get(k): memtable first, then newest file to oldest. A Bloom filter says 'definitely not here' so most files are never read.

STORAGE ENGINES: how an LSM-tree turns random writes into sequential ones

	B-tree	LSM-tree
Writes	update pages in place (plus a WAL)	append to log and memtable, merge later
Reads	predictable: a few page reads	check memtable and several files (Bloom filters help)
Costs	random writes, page splits	compaction work, write and space amplification
Best for	read-heavy, transactional workloads	write-heavy workloads and very large datasets

The LSM write path in miniature

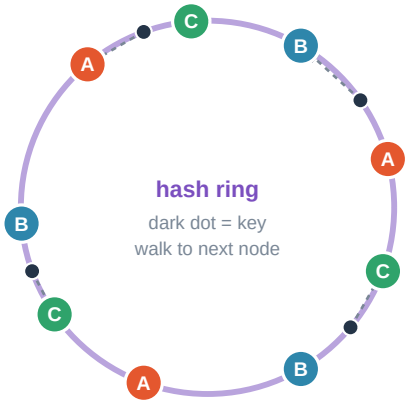
```
class MiniLsm {
public:
    explicit MiniLsm(const std::string& wal) : wal_(wal, std::ios::binary | std::ios::app) {}
    void put(const std::string& k, const std::string& v) {
        wal_ << k.size() << ' ' << v.size() << '\n' << k << v;   // 1) sequential append: cheap and crash-safe
        wal_.flush();  //    real engines fsync, usually with group commit
        bytes_ += k.size() + v.size();
        mem_[k] = v;                                             // 2) sorted in-memory table
        if (bytes_ >= kFlushBytes) flush_to_sstable();  // 3) immutable sorted file, then rotate the WAL
    }
    std::optional<std::string> get(const std::string& k) const {
        if (auto it = mem_.find(k); it != mem_.end()) return it->second;
        return search_sstables_newest_first(k);                  // Bloom filter per table skips most disk reads
    }
private:
    void flush_to_sstable();
    std::optional<std::string> search_sstables_newest_first(const std::string&) const;
    std::map<std::string, std::string> mem_;  std::ofstream wal_;  std::size_t bytes_ = 0;
    static constexpr std::size_t kFlushBytes = 64u << 20;
};
```

Replication lag creates three anomalies you must design around

Read-your-writes
You save a profile, refresh, and see the old one. Read your own data from the leader.

Monotonic reads
You see a comment, refresh, it vanishes. Pin a user to one replica.

Consistent prefix
An answer appears before its question. Keep causally related writes in order.



Add a 5th server: how many keys have to move?

hash(key) % N

about 80% of keys move

consistent hashing (with virtual nodes)

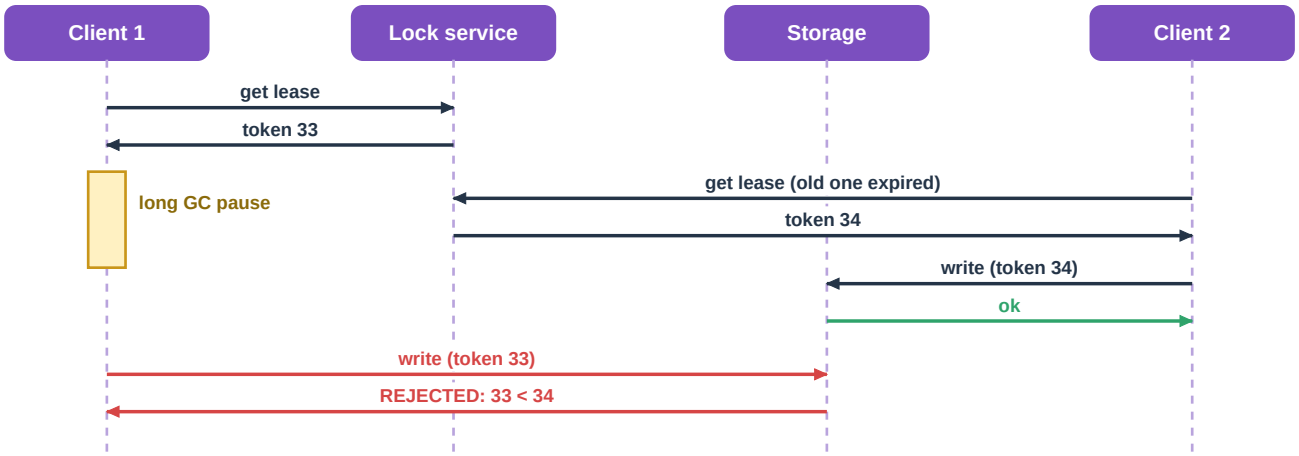
about 20% (1/N)

Each server appears many times on the ring (virtual nodes) so load spreads evenly and a removed node's keys scatter across many peers.

PARTITIONING: never place data with hash(key) % N, use a ring

Consistent hashing with virtual nodes

```
class HashRing {
public:
    explicit HashRing(int vnodes = 128) : vnodes_(vnodes) {}
    void add(const std::string& n)    { for (int i = 0; i < vnodes_; ++i) ring_[h(n + "#" + std::to_string(i))] = n; }
    void remove(const std::string& n) {
        for (int i = 0; i < vnodes_; ++i) ring_.erase(h(n + "#" + std::to_string(i)));
    }
    const std::string& owner(const std::string& key) const {         // precondition: ring is not empty
        auto it = ring_.lower_bound(h(key));
        return (it == ring_.end() ? ring_.begin() : it)->second;     // wrap around the ring
    }
private:
    static std::uint64_t h(const std::string& s) { return fnv1a64(s); }  // stable across processes, unlike std::hash
    std::map<std::uint64_t, std::string> ring_;  int vnodes_;
};
```



DISTRIBUTED TROUBLE: a paused process still believes it holds the lock. Fencing tokens catch it

Fencing token check, an idempotent event consumer, and a queue with backpressure

```
class FencedStore {                                          // reject writers that hold an old lease
public:
    bool write(std::uint64_t token, const std::string& k, std::string v) {
        if (token < max_token_) return false;  // the paused ex-leader wakes up with a stale token
        max_token_ = token;  data_[k] = std::move(v);  return true;
    }
private:
    std::uint64_t max_token_ = 0;  std::unordered_map<std::string, std::string> data_;
};

class Account {  // event-sourced, idempotent: replays are harmless
public:
    bool apply(const Event& e) {                             // assumes in-order delivery per key
        if (e.seq <= last_seq_) return false;                // duplicate or replay: ignore
        balance_ += (e.type == "deposit" ? 1 : -1) * e.amount_cents;
        last_seq_ = e.seq;  return true;  // persist balance_ and last_seq_ in ONE atomic write
    }
private:
    std::int64_t balance_ = 0;  std::uint64_t last_seq_ = 0;
};

template <class T> class BoundedQueue {  // producers block instead of growing memory forever
public:
    explicit BoundedQueue(std::size_t cap) : cap_(cap) {}
    void push(T v) { std::unique_lock lk(m_); not_full_.wait(lk, [&] { return q_.size() < cap_; });
                     q_.push_back(std::move(v)); not_empty_.notify_one(); }
    T pop()        { std::unique_lock lk(m_); not_empty_.wait(lk, [&] { return !q_.empty(); });
                     T v = std::move(q_.front()); q_.pop_front(); not_full_.notify_one(); return v; }
private:  // production code also needs close() and timeouts
    std::mutex m_;  std::condition_variable not_full_, not_empty_;  std::deque<T> q_;  std::size_t cap_;
};
```

WATCH OUT

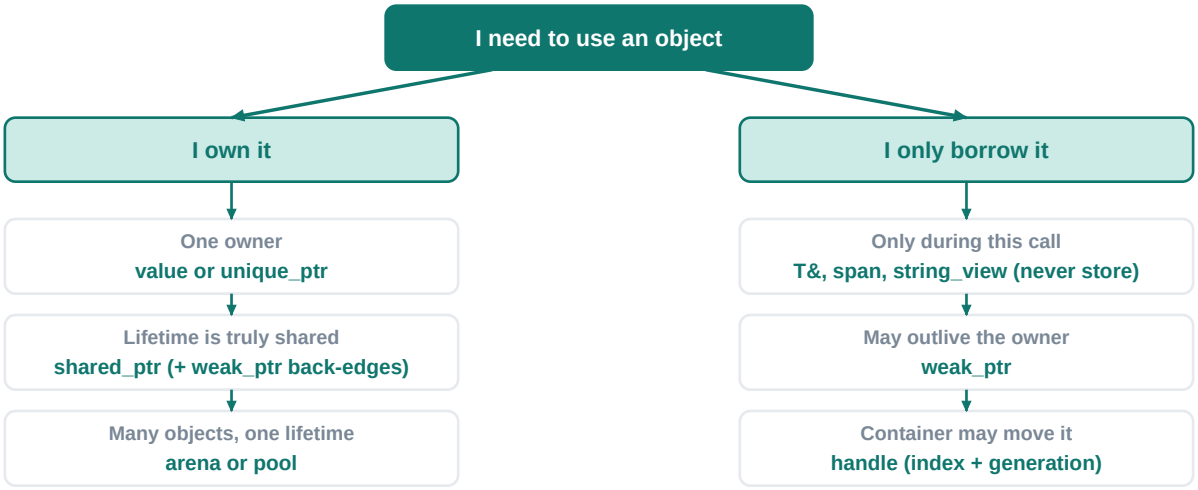
- **Dual writes** (database plus cache or search index) drift apart silently. Write once to a log and derive the rest.
- **Wall-clock last-write-wins** loses data when clocks skew; **retrying a non-idempotent call** double-charges.
- **Snapshot isolation is not serializable:** write skew lets two on-call doctors both go off call.
- **Unbounded queues** turn a slow consumer into an out-of-memory crash; **hot keys** defeat hash partitioning; LSM deletes need tombstones, and tombstones need compaction.

MEMORY TIP

Bound everything: memtable, block cache (LRU with a byte budget), queue depth, window state. Allocate memtable entries from an arena freed at flush. A Bloom filter at roughly 10 bits per key buys about 1% false positives. `mmap` hides page-cache pressure from your allocator and turns I/O errors into signals. Zero-copy `string_view` parsing ties every view to its buffer: copy before the buffer is recycled.

5 Memory Management Playbook

C++ ownership across all four books | decide who owns it, who borrows it, and when it dies



OWNERSHIP DECISION TREE: pick the simplest tool that fits

Rule of Five, an arena per request, and four classic lifetime bugs

```
class Buffer {                                              // raw resource => define all five (or none)
public:
    explicit Buffer(std::size_t n) : data_(new std::byte[n]), size_(n) {}
    ~Buffer() { delete[] data_; }
    Buffer(const Buffer& o) : data_(new std::byte[o.size_]), size_(o.size_)
    { std::memcpy(data_, o.data_, size_); }
    Buffer(Buffer&& o) noexcept
        : data_(std::exchange(o.data_, nullptr)), size_(std::exchange(o.size_, 0)) {}
    Buffer& operator=(Buffer o) noexcept { swap(o); return *this; }      // copy-and-swap
    void swap(Buffer& o) noexcept { std::swap(data_, o.data_); std::swap(size_, o.size_); }
private:
    std::byte* data_;  std::size_t size_;
};  // Better: std::vector<std::byte> => Rule of Zero

alignas(std::max_align_t) static thread_local std::byte scratch[1 << 20];
void handle_request(const Request& r) {
    std::pmr::monotonic_buffer_resource arena{scratch, sizeof scratch};  // falls back to the heap if exhausted
    std::pmr::vector<std::pmr::string> parts{&arena};  // every allocation comes from the arena
    split(r.body, parts);  process(parts);
}  // arena dies: everything released at once, O(1)

std::string_view name() { std::string s = load(); return s; }               // BUG 1: view of a destroyed local
struct Node { std::shared_ptr<Node> next; std::weak_ptr<Node> prev; };      // BUG 2 avoided: weak back-edge
auto& first = v[0];  v.push_back(x);  // BUG 3: 'first' dangles after reallocation
// Conn c; c.start();  // start() calls shared_from_this()   // BUG 4: bad_weak_ptr; use make_shared
```

Share by immutability
`shared_ptr<const T>` snapshots swapped atomically, or copy-on-write. A data race is undefined behaviour, not just a stale read.

Or pass messages
Bounded queues between threads. Take multiple locks with `std::scoped_lock`, in a fixed order.

Lock-free is a last resort
Needs correct `memory_order` and an ABA plan. Prefer one pool per thread until profiling says otherwise.

Async and `this`
Callbacks that capture `this` must hold `shared_from_this()` or a `weak_ptr` so the object outlives the operation.

Sanitizers in CI
ASan (memory errors, leaks), UBSan, TSan (races), MSan (uninitialised reads). Run tests under ASan and UBSan on every commit.

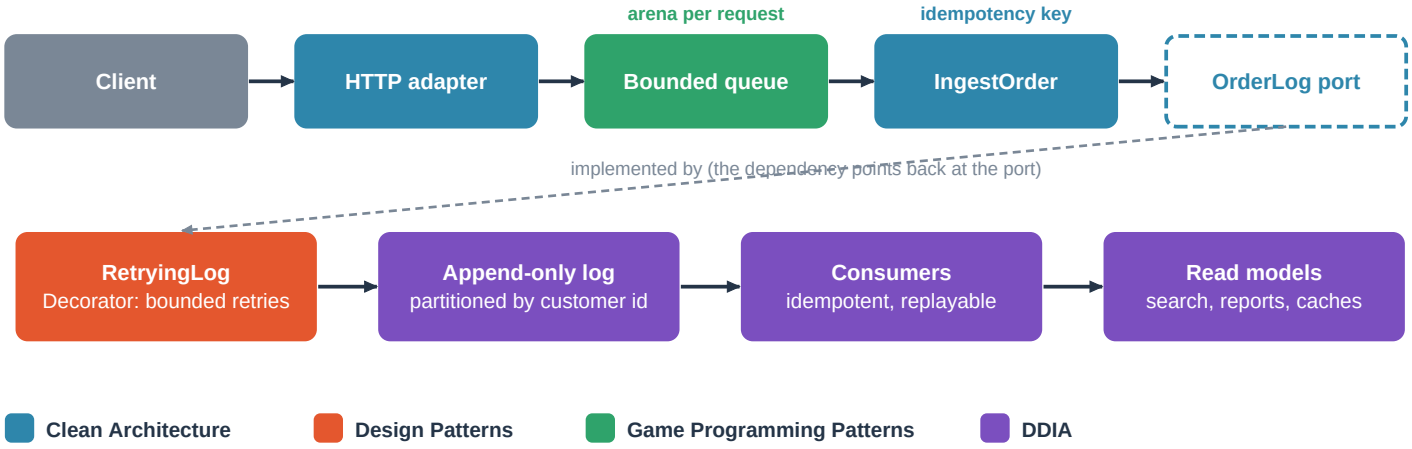
Profile memory too
heaptrack, Valgrind massif, allocator stats from jemalloc, tcmalloc or mimalloc. Fragmentation and peak RSS are the usual production problem.

Warnings and analysis
`-Wall -Wextra -Wshadow -Wconversion`, clang-tidy (bugprone, cppcoreguidelines), debug standard-library modes in test builds.

Make leaks structural
No owning raw pointers, no naked `new`/`delete`, RAII wrappers for files, sockets, locks and GPU handles.

6 Putting It All Together

One order-ingest service that must never lose or double-process an order



ONE REQUEST, FOUR LENSES

The pieces meet: a port, a retrying Decorator (safe only because append is idempotent), and the use case

```
struct OrderLog {                                              // port declared by the use-case layer
    virtual ~OrderLog() = default;
    virtual bool append(std::string_view idempotency_key, std::string_view payload) = 0;  // same key => same effect
};

class RetryingLog final : public OrderLog {                    // GoF Decorator: owns what it wraps
public:
    RetryingLog(std::unique_ptr<OrderLog> inner, int attempts) : inner_(std::move(inner)), attempts_(attempts) {}
    bool append(std::string_view key, std::string_view payload) override {
        for (int i = 0; i < attempts_; ++i) {
            if (inner_->append(key, payload)) return true;
            std::this_thread::sleep_for(std::chrono::milliseconds(10 << i));          // exponential backoff
        }
        return false;                                          // bounded retries: surface the failure
    }
private:
    std::unique_ptr<OrderLog> inner_;  int attempts_;
};

class IngestOrder {                                            // Clean Architecture interactor
public:
    explicit IngestOrder(OrderLog& log) : log_(log) {}
    bool operator()(const OrderRequest& r) { return log_.append(r.idempotency_key, serialize(r)); }
private:
    OrderLog& log_;
};
// main: RetryingLog log{std::make_unique<KafkaLog>(cfg), 4};  IngestOrder ingest{log};
```

Why it holds together: retries are safe because the operation is idempotent (DDIA); the retry policy can change without touching business rules because it is a decorator behind a port (GoF plus Clean Architecture); memory stays bounded because queues are capped and parsing uses arenas (Game Patterns); and every object has exactly one owner. When something breaks you can name which of the four concerns failed.

7 Cheat Sheet

The eight traps that bite hardest, and which pattern to reach for

1 Global state in disguise
Singleton, Service Locator. Pass dependencies from `main` instead.

2 Dependencies pointing outward
Business code includes DB or UI headers. Define ports in the inner layer.

3 Over-engineering
Five files to add one field. Add layers only where change is real.

4 Dangling pointers
Observer, Command, components. Use weak refs, RAII subscriptions, handles.

5 Unbounded growth
Queues, caches, windows. Bound them and decide the overflow policy.

6 Allocation in the hot path
Latency spikes and fragmentation. Pools, arenas, `reserve`.

7 Trusting clocks and networks
Use idempotency, fencing tokens, logical clocks, consensus.

8 Dual writes
Systems drift apart. One source of truth plus a log or outbox.

If your problem is...	Reach for...	From
Picking a concrete type at runtime	Factory Method with a registry, or `std::variant`	GoF
Adding behaviour without subclass explosion	Decorator, Strategy, policy templates	GoF
Swapping the database or framework later	Ports and adapters, composition root	Clean Arch.
Faster builds, independent deployment	No dependency cycles, pimpl, stable abstractions	Clean Arch.
Predictable frame or request latency	Fixed step, arenas, pools, struct-of-arrays	Game Patterns
High write throughput, durable storage	LSM-tree with WAL and Bloom filters	DDIA
Scaling out and surviving failures	Consistent hashing, quorums, fencing tokens	DDIA
Keeping derived views consistent	Append-only log, idempotent consumers, outbox	DDIA

KEY IDEA

Six-question checklist. (1) Can you state every object's owner and lifetime in one sentence? (2) Do dependencies point toward the business rules? (3) Is every queue, cache and buffer bounded? (4) Is every retried operation idempotent? (5) Do ASan, UBSan and TSan run in CI? (6) Did you measure before optimising? If yes to all six, the exact pattern you picked matters far less than you fear.